

附件 3

西安电子科技大学

智能系统专业实验 课程实验报告

实验名称 实验 4-数字识别

XXXXXXXX 学院 2320xxx 班

姓名 XXXXXX 学号 XXXXXXXXXXXX

同作者 XXXXXXXX

实验日期 202x 年 x 月 x 日

成 绩

实验报告内容基本要求及参考格式

- 一、实验目的
- 二、实验环境
- 三、实验基本原理及步骤（模型描述、理论描述或方案描述）
- 四、实验自主创新（或改进）部分的描述
- 五、实验结果与分析（需给出结果图或表，并进行分析）
- 六、实验的收获及心得体会（或遇到的问题，如何解决）
- 七、实验代码

一、实验目的

- 1. 了解 *MNIST* 手写数字数据集的结构与加载方式。
- 2. 掌握 *LeNet-5* 卷积神经网络的架构及其实现。
- 3. 使用 *PyTorch* 框架完成手写数字识别任务，并评估模型性能。

二、实验环境

- 1. 硬件: SUB KIT NX 嵌入式开发板
- 2. 操作系统: Linux
- 3. 软件及库:
 - 1) Python 3
 - 2) NumPy
 - 3) PyTorch
 - 4) Matplotlib

三、实验基本原理及步骤（模型描述、理论描述或方案描述）

1. 基本原理

MNIST 数据集包含 28×28 像素的灰度手写数字图像（0~9），共 60000 张训练图片和 10000 张测试图片。本实验采用 *LeNet-5* 卷积神经网络，其结构如下：

层	类型	输入尺寸	输出尺寸	参数
1	<i>Conv2d</i> ($1 \rightarrow 6, 5 \times 5, padding = 2$) + <i>ReLU</i>	$1 \times 28 \times 28$	$6 \times 28 \times 28$	保持尺寸
2	<i>MaxPool2d</i> ($2 \times 2, stride = 2$)	$6 \times 28 \times 28$	$6 \times 14 \times 14$	降采样
3	<i>Conv2d</i> ($6 \rightarrow 16, 5 \times 5$) + <i>ReLU</i>	$6 \times 14 \times 14$	$16 \times 10 \times 10$	无 <i>padding</i>
4	<i>MaxPool2d</i> ($2 \times 2, stride = 2$)	$16 \times 10 \times 10$	$16 \times 5 \times 5$	降采样
5	<i>Flatten</i> + <i>Linear</i> ($16 \times 5 \times 5 = 400 \rightarrow 120$) + <i>ReLU</i>	400	120	全连接
6	<i>Linear</i> ($120 \rightarrow 84$) + <i>ReLU</i>	120	84	全连接
7	<i>Linear</i> ($84 \rightarrow 10$)	84	10	输出 <i>logits</i>

- 1) 损失函数: 交叉熵 (*CrossEntropyLoss*)。
- 2) 优化算法: *SGD* (学习率 0.01, 动量 0.5)

2. 实验步骤

1) 数据加载与预处理

- a) 使用 *torchvision.datasets.MNIST* 自动下载数据集。
- b) 预处理: *ToTensor()* 将像素值归一化到 [0,1], 再使用均值 0.1307、标准差 0.3081 进行标准化。
- c) 创建 *DataLoader*: 训练集 *batch_size* = 64, 打乱顺序; 测试集 *batch_size* = 1000。

2) 构建 *LeNet-5* 网络

- a) 按上述结构定义 *LeNet5* 类, 继承 *nn.Module*
- b) 前向传播中依次经过卷积、*ReLU*、池化、展平、全连接

3) 训练模型

- a) 共 10 个 *epoch*, 每个 *epoch* 内迭代所有训练 *batch*
- b) 计算损失、反向传播、更新参数
- c) 每 10 个 *batch* 打印一次训练损失

4) 测试模型

- a) 每个 *epoch* 结束后在测试集上评估平均损失和准确率
- b) 保存测试准确率最高的模型权重

5) 可视化

使用 *Matplotlib* 绘制双 *y* 轴曲线: 左轴为训练平均损失, 右轴为测试准确率, 横轴为 *epoch*

四、实验自主创新 (或改进) 部分的描述

1. 增加训练损失记录与可视化

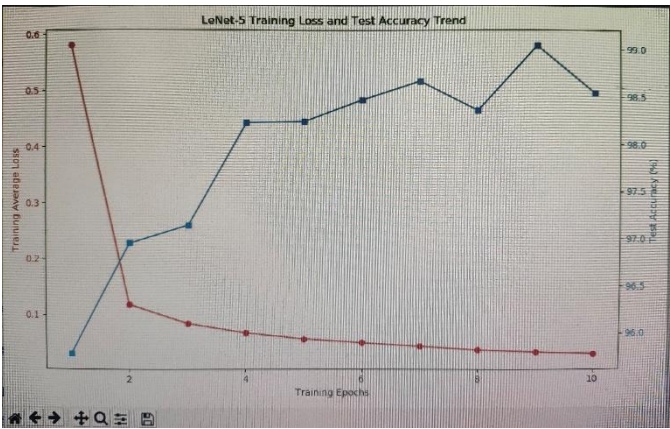
原始实验指导书只打印了部分训练信息, 本实验在每个 *epoch* 结束时记录平均训练损失, 并与测试准确率在同一张图上对比显示, 便于分析损失下降与准确率上升的关系。

2. 设置随机种子

通过 *torch.manual_seed(42)* 固定随机种子, 使实验结果可复现。

五、实验结果与分析（需给出结果图或表，并进行分析）

1. 最终拟合效果图



- 1) 红色曲线（训练损失）：快速下降，前 2 个 *epoch* 降幅最大，之后缓慢下降并趋于平缓。
- 2) 蓝色曲线（测试准确率）：稳步上升伴有轻微震荡，第 1 个 *epoch* 已达 95%，后续逐渐提升至 99.1%。
2. 损失变化：失函数平稳下降，未出现震荡或发散，说明学习率选择合适，网络结构能够表达目标函数。最终损失稳定在0.0103。

六、实验的收获及心得体会（或遇到的问题，如何解决）

1. 掌握了 **LeNet-5** 的完整实现：理解了卷积层、池化层、全连接层的参数计算（特别是特征图尺寸的变化），并能够用 *PyTorch* 正确搭建。
2. 数据预处理的重要性：*MNIST* 的标准化参数 (0.1307,0.3081) 是事先计算好的，使用这些系数可以加速收敛。若不进行标准化，训练损失可能下降更慢。
3. 训练过程可视化对调参的帮助：通过观察损失曲线和准确率曲线，可以快速判断学习率是否合适（损失震荡可能表示学习率过大）、是否过拟合（验证准确率下降）。

七、实验代码

代码
<pre>import torch import torch.nn as nn import torch.nn.functional as F import torch.optim as optim from torchvision import datasets, transforms</pre>

```

from torch.utils.data import DataLoader
import matplotlib.pyplot as plt

torch.manual_seed(42)

# 1. 数据加载和预处理
def load_data():
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.1307,), (0.3081,))
    ])
    train_dataset = datasets.MNIST(root='./', train=True,
download=True, transform=transform)
    test_dataset = datasets.MNIST(root='./', train=False,
download=True, transform=transform)
    train_loader = DataLoader(train_dataset, batch_size=64,
shuffle=True, num_workers=2)
    test_loader = DataLoader(test_dataset, batch_size=1000,
shuffle=False, num_workers=2)
    return train_loader, test_loader

# 2. LeNet-5 网络
class LeNet5(nn.Module):
    def __init__(self):
        super(LeNet5, self).__init__()
        self.conv1 = nn.Conv2d(1, 6, kernel_size=5, padding=2)
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5)
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = self.pool1(x)
        x = F.relu(self.conv2(x))
        x = self.pool2(x)

```

```

x = x.view(-1, 16 * 5 * 5)
x = F.relu(self.fc1(x))
x = F.relu(self.fc2(x))
x = self.fc3(x)
return x

```

3. 训练函数

```

def train(model, device, train_loader, optimizer, epoch,
log_interval=10, epoch_loss_list=None):
    model.train()
    criterion = nn.CrossEntropyLoss()
    total_epoch_loss = 0.0
    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        total_epoch_loss += loss.item()
        loss.backward()
        optimizer.step()
        if batch_idx % log_interval == 0:
            print(f'Train Epoch: {epoch} [{batch_idx *
len(data)} / {len(train_loader.dataset)}]
                f'({100. * batch_idx /
len(train_loader):.0f}%) \t Loss: {loss.item():.6f}')
            avg_epoch_loss = total_epoch_loss / len(train_loader)
            epoch_loss_list.append(avg_epoch_loss)

```

4. 测试函数

```

def test(model, device, test_loader, acc_list=None):
    model.eval()
    test_loss = 0
    correct = 0
    criterion = nn.CrossEntropyLoss(reduction='sum')
    with torch.no_grad():
        for data, target in test_loader:
            data, target = data.to(device), target.to(device)
            output = model(data)

```

```

        test_loss += criterion(output, target).item()
        pred = output.argmax(dim=1, keepdim=True)
        correct += pred.eq(target.view_as(pred)).sum().item()

    test_loss /= len(test_loader.dataset)
    accuracy = 100. * correct / len(test_loader.dataset)
    print(f'\nTest set: Average loss: {test_loss:.4f}, '
          f'Accuracy: {correct} / {len(test_loader.dataset)}'
          (f'{accuracy:.2f}%')\n')
    if acc_list is not None:
        acc_list.append(accuracy)
    return accuracy

# 5. 主函数
def main():
    device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
    print(f"使用设备: {device}")
    train_loader, test_loader = load_data()
    model = LeNet5().to(device)
    optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.5)

    epochs = 10
    log_interval = 10
    epoch_loss_list = []
    accuracy_list = []

    best_accuracy = 0
    for epoch in range(1, epochs + 1):
        train(model, device, train_loader, optimizer, epoch,
log_interval, epoch_loss_list)
        accuracy = test(model, device, test_loader, accuracy_list)
        if accuracy > best_accuracy:
            best_accuracy = accuracy
            torch.save(model.state_dict(), 'best_lenet5_model.pth')
            print(f"保存最佳模型, 准确率: {accuracy:.2f}%")
    print(f"训练完成! 最佳测试准确率: {best_accuracy:.2f}%")

# 绘制双轴折线图

```

```
plt.rcParams['font.sans-serif'] = ['DejaVu Sans']
fig, ax1 = plt.subplots(figsize=(10, 6))
ax1.set_xlabel('Training Epochs')
ax1.set_ylabel('Training Average Loss', color='tab:red')
ax1.plot(range(1, epochs+1), epoch_loss_list, color='tab:red',
marker='o', label='Loss')
ax1.tick_params(axis='y', labelcolor='tab:red')

ax2 = ax1.twinx()
ax2.set_ylabel('Test Accuracy (%)', color='tab:blue')
ax2.plot(range(1, epochs+1), accuracy_list, color='tab:blue',
marker='s', label='Accuracy')
ax2.tick_params(axis='y', labelcolor='tab:blue')

plt.title('LeNet-5 Training Loss and Test Accuracy Trend')
fig.tight_layout()
plt.show()

if __name__ == '__main__':
    main()
```